

Functions and Recursion in C

📌 Definition to Remember

A function is a self-contained, modular block of code performing a specific task. Recursion occurs when a function calls itself directly or indirectly until reaching a base termination condition.

1. Concept Overview & Fundamentals 🗨️

Functions and Recursion in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Code is compiled directly into native machine instructions without virtual machine overhead.
- Memory is strictly partitioned into **Code/Text Segment**, **Data Segment (Initialized/BSS)**, **Stack**, and **Heap**.
- Direct pointer and register manipulation provides deterministic performance critical for operating systems, embedded hardware, and database kernels.

2. Technical Architecture & Memory Mechanics ⚙️

Memory Layout & Storage Organization

```
+-----+
|                STACK SEGMENT                |
| - Local variables, function call stack frames, parameters |
| - High memory growing downwards towards Heap |
+-----+
|
| v
| ^
|
+-----+
|                HEAP SEGMENT                |
| - Dynamic memory allocations via malloc(), calloc() |
| - Low memory growing upwards towards Stack |
+-----+
|                INITIALIZED DATA SEGMENT    |
| - Global and static variables with non-zero initial values |
+-----+
|                BSS SEGMENT (UNINITIALIZED)  |
| - Global and static variables initialized to zero |
+-----+
|                CODE / TEXT SEGMENT          |
| - Read-only executable binary instructions |
+-----+
```

Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers dynamic scalability	Stack: O(1) push/pop; Heap: O(1) alloc
Type Checking	Static Typing	Compiler verifies types during compilation	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memory address	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops, and call stacks	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Function and Recursion\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.
- **Embedded & IoT Devices:** Microcontrollers with constrained RAM (8KB-64KB) require C's predictable footprint.
- **Database Engine Kernels:** High-throughput storage engines (PostgreSQL, SQLite, MySQL InnoDB) rely on low-level buffer pool managers written in C.
- **Game Engine Foundations:** Graphics rasterization, physics engines, and GPU interfaces utilize C/C++ memory arrays for zero-overhead computation.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: add function

Problem Description & Objective:

Implementation of add function

C Code Implementation:

```
/*
Write a function that adds that takes 4 int
parameters and returns the sum.
*/

#include <stdio.h>

// Function Definition
int add(int a, int b, int c, int d) {
    return a + b + c + d;
}

int main() {
    /*
    * Logic:
    * - Goal: Write a function that adds that takes 4 int parameters and returns the sum.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int result;

    result = add(10, 20, 30, 40);

    printf("Sum = %d\n", result);

    return 0;
}
```

Problem 2: fibonacci recursion

Problem Description & Objective:

Implementation of fibonacci recursion

C Code Implementation:

```

/*
Create a program using recursion to display
the Fibonacci series upto a certain number.
*/

#include <stdio.h>

// Tail-recursive helper function to calculate Fibonacci in O(n) time
int fibonacciTail(int n, int a, int b) {
    /*
    * Logic:
    * 1. Uses Tail Recursion to calculate the n-th Fibonacci number.
    * 2. The parameters 'a' and 'b' act as running accumulators for F(i) and F(i+1).
    * 3. Base Cases: if n == 0, returns 'a'. If n == 1, returns 'b'.
    * 4. Recursive Step: Calls fibonacciTail(n - 1, b, a + b).
    * Since the recursive call is the final statement, modern compilers optimize it to a simple loop.
    * This improves time complexity from exponential O(2^n) to linear O(n), returning answers
    instantly.
    */
    if (n == 0) {
        return a;
    }
    if (n == 1) {
        return b;
    }
    return fibonacciTail(n - 1, b, a + b);
}

// Recursive Function Wrapper
int fibonacci(int n) {
    if (n < 0) {
        return 0; // Guard against negative inputs
    }
    return fibonacciTail(n, 0, 1);
}

int main() {
    int n, i;

    printf("Enter number of terms: ");
    if (scanf("%d", &n) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    if (n < 0) {
        printf("Number of terms cannot be negative.\n");
        return 1;
    }

    printf("Fibonacci Series:\n");

    for(i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }
    printf("\n");
}

```

```
    return 0;
}
```

Problem 3: get average function

Problem Description & Objective:

Implementation of get average function

C Code Implementation:

```
/*
Call a function get_average that takes five int
numbers and returns the average.
*/

#include <stdio.h>

// Function Definition
float get_average(int a, int b, int c, int d, int e) {

    return (a + b + c + d + e) / 5.0;
}

int main() {
    /*
    * Logic:
    * - Goal: Call a function get_average that takes five int numbers and returns the average.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    float average;

    average = get_average(10, 20, 30, 40, 50);

    printf("Average = %.2f\n", average);

    return 0;
}
```

Problem 4: greet function

Problem Description & Objective:

Implementation of greet function

C Code Implementation:

```
/*
Write a function named greet that prints
"Hello, World!" when called.
*/

#include <stdio.h>

// Function Definition
void greet() {
    printf("Hello, World!\n");
}

int main() {
    /*
    * Logic:
    * - Goal: Write a function named greet that prints "Hello, World!" when called.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    // Function Call
    greet();

    return 0;
}
```

Problem 5: increment function

Problem Description & Objective:

Implementation of increment function

C Code Implementation:

```

/*
Demonstrate with a function increment that the
original integer passed to it does not change
after incrementing it inside the function.
*/

#include <stdio.h>

// Function Definition
void increment(int num) {
    num++;

    printf("Value inside function = %d\n", num);
}

int main() {
    /*
    * Logic:
    * - Goal: Demonstrate with a function increment that the original integer passed to it does not
change after incrementing it inside the function.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int number = 10;

    printf("Original value before function call = %d\n", number);

    increment(number);

    printf("Original value after function call = %d\n", number);

    return 0;
}

```

Problem 6: max function

Problem Description & Objective:

Implementation of max function

C Code Implementation:

```

/*
Create a function max that takes two float
arguments and returns the larger value.
*/

#include <stdio.h>

// Function Definition
float max(float a, float b) {

    if(a > b) {
        return a;
    } else {
        return b;
    }
}

int main() {
    /*
    * Logic:
    * - Goal: Create a function max that takes two float arguments and returns the larger value.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    float num1, num2, result;

    printf("Enter first number: ");
    scanf("%f", &num1);

    printf("Enter second number: ");
    scanf("%f", &num2);

    result = max(num1, num2);

    printf("Larger number = %.2f\n", result);

    return 0;
}

```

Problem 7: palindrome recursion

Problem Description & Objective:

Implementation of palindrome recursion

C Code Implementation:

```

/*
Create a program using recursion to check
if a number is a palindrome using recursion.
*/

#include <stdio.h>

// Recursive Function
int reverseNumber(int num, int rev) {

    if(num == 0) {
        return rev;
    }

    return reverseNumber(num / 10, rev * 10 + num % 10);
}

int main() {
    /*
    * Logic:
    * - Goal: Create a program using recursion to check if a number is a palindrome using recursion.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int num, reversed;

    printf("Enter a number: ");
    scanf("%d", &num);

    reversed = reverseNumber(num, 0);

    if(num == reversed) {
        printf("%d is a Palindrome Number.\n", num);
    } else {
        printf("%d is NOT a Palindrome Number.\n", num);
    }

    return 0;
}

```

Problem 8: print date function

Problem Description & Objective:

Implementation of print date function

C Code Implementation:

```

/*
Call a function print_date that prints the current date.
Define the function without any parameters.
*/

#include <stdio.h>

// Function Definition
void print_date() {
    printf("Current Date: 01/06/2026\n");
}

int main() {
    /*
    * Logic:
    * - Goal: Call a function print_date that prints the current date. Define the function without any
parameters.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    // Function Call
    print_date();

    return 0;
}

```

Problem 9: square function

Problem Description & Objective:

Implementation of square function

C Code Implementation:

```

/*
Define a function square that takes an int
and returns its square.
*/

#include <stdio.h>

// Function Definition
int square(int num) {
    return num * num;
}

int main() {
    /*
    * Logic:
    * - Goal: Define a function square that takes an int and returns its square.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int number, result;

    printf("Enter a number: ");
    scanf("%d", &number);

    result = square(number);

    printf("Square = %d\n", result);

    return 0;
}

```

6. Exam Scoring Points & Memory Revision 🎯

★ Must-Write Points

1. Function structure: Return type, function name, parameter list, and function body.
2. Pass-by-value copies data to local parameters; changes do not affect caller arguments.
3. Pass-by-reference in C is simulated by passing pointer addresses (&var).
4. Recursion requirements: Base case (termination condition) and Recursive step (approaching base).
5. Call Stack mechanics: Each recursive call pushes an Activation Record / Stack Frame.

⚡ Quick Recall

Prototype Declaration -> Call Stack Push -> Parameter Binding -> Base Case Test -> Return Unwind